

操作系统实验报告

Lab4:进程管理

信息安全 2313781 李胜林 2312796 张肇秋 2312323 杨中秀

一、实验目的

1. 了解虚拟内存管理的基本结构，掌握虚拟内存的组织与管理方式
2. 了解内核线程创建/执行的管理过程
3. 了解内核线程的切换和基本调度过程

二、实验内容

本次实验将在实验二和实验三的基础上进行，主要是完成以下两方面的内容：其一是**进一步完善虚拟内存管理，实现基本的地址空间结构**。通过引入虚拟内存描述结构，管理进程或线程的虚拟地址空间布局，为每个执行实体提供逻辑上的运行空间。其二是**引入内核线程机制，实现多执行流并发运行能力**。实现线程控制块、上下文切换和调度器等内容，从而实现多线程并发执行，使内核能够调度多个执行实体轮流使用 CPU 运行。

三、练习

(一) 练习0：填写已有实验

本实验依赖实验2/3。请把你做的实验2/3的代码填入本实验中代码中有“LAB2”,“LAB3”的注释相应部分。

(二) 练习1：分配并初始化一个进程控制块（需要编码）

`alloc_proc` 函数（位于 `kern/process/proc.c` 中）负责分配并返回一个新的 `struct proc_struct` 结构，用于存储新建立的内核线程的管理信息。`ucore` 需要对这个结构进行最基本的初始化，你需要完成这个初始化过程。

请在实验报告中简要说明你的设计实现过程。并说明 `proc_struct` 中 `struct context context` 和 `struct trapframe *tf` 成员变量含义和在本实验中的作用是什么？

(三) 练习2：分配并初始化一个进程控制块（需要编码）

完成在 `kern/process/proc.c` 中的 `do_fork` 函数中的处理过程。它的大致执行步骤包括：

1. 调用 `alloc_proc`，首先获得一块用户信息块。
2. 为进程分配一个内核栈。
3. 复制原进程的内存管理信息到新进程
4. 复制原进程上下文到新进程
5. 将新进程添加到进程列表
6. 唤醒新进程
7. 返回新进程号

请在实验报告中简要说明你的设计实现过程，并回答 `ucore` 是否做到给每个新 `fork` 的线程一个唯一的 `id`

(四) 练习3：编写 `proc_run` 函数（需要编码）

`proc_run` 用于将指定的进程切换到CPU上运行。它的大致执行步骤包括：

1. 检查要切换的进程是否与当前正在运行的进程相同，如果相同则不需要切换。
2. 禁用中断。使用宏 `local_intr_save(x)` 和 `local_intr_restore(x)` 来实现关、开中断。
3. 切换当前进程为要运行的进程。
4. 切换页表，以便使用新进程的地址空间。使用 `lsatp(unsigned int pgdir)` 函数，可实现修改 SATP寄存器值的功能。
5. 实现上下文切换，使用 `switch_to()` 函数。可实现两个进程的context切换。
6. 允许中断

请回答在本实验的执行过程中，创建且运行了几个内核线程？

(五) 扩展练习Challenge

1. 说明语句 `local_intr_save(intr_flag);...local_intr_restore(intr_flag);` 是如何实现开关中断的？
2. 深入理解不同分页模式的工作原理（思考题）

`get_pte()` 函数（位于 `kern/mm/pmm.c`）用于在页表中查找或创建页表项，从而实现对指定线性地址对应的物理页的访问和映射操作。这在操作系统中的分页机制下，是实现虚拟内存与物理内存之间映射关系非常重要的内容。

- `get_pte()` 函数中有两段形式类似的代码，结合 `sv32`，`sv39`，`sv48` 的异同，解释这两段代码为什么如此相像。
- 目前 `get_pte()` 函数将页表项的查找和页表项的分配合并在一个函数里，你认为这种写法好吗？有没有必要把两个功能拆开？

四、练习解答

(一) 练习0：填写已有实验

我们只需要在文件 `lab4/kern/trap/trap.c` 中补充我们先前的时间中断的代码即可，如下所示：

```
1  #include <sbi.h>
2
3  static size_t num = 0;
4
5  /*LAB3 请补充你在lab3中的代码 */
6  clock_set_next_event();
7  ticks++;
8  if(ticks%TICK_NUM==0){
9      print_ticks();
10     num++;
11     if(num>=10){
12         sbi_shutdown();
13     }
14 }
```

(二)练习1：分配并初始化一个进程控制块

编码

在这个练习中，我们需要实现的是 `alloc_proc` 函数，这个函数主要是实现分配并初始化进程控制块(`proc_struct`)功能。它的核心功能是从内核堆中分配一块内存用于存储**进程控制块** (`struct proc_struct`)，并对 PCB 中的所有字段进行默认**初始化**，返回一个“未就绪但结构完整”的**进程控制块指针**。

而我们可以对应的头文件中找到该进程控制块的声明：

```
1 struct proc_struct
2 {
3     enum proc_state state;           // Process state
4     int pid;                         // Process ID
5     int runs;                        // the running times of Proces
6     uintptr_t kstack;               // Process kernel stack
7     volatile bool need_resched;     // bool value: need to be rescheduled to
    release CPU?
8     struct proc_struct *parent;      // the parent process
9     struct mm_struct *mm;           // Process's memory management field
10    struct context context;          // Switch here to run process
11    struct trapframe *tf;            // Trap frame for current interrupt
12    uintptr_t pgdir;                // the base addr of Page Directroy
    Table(PDT)
13    uint32_t flags;                  // Process flag
14    char name[PROC_NAME_LEN + 1];    // Process name
15    list_entry_t list_link;          // Process link list
16    list_entry_t hash_link;          // Process hash list
17 };
```

这里面有几个重要的成员变量：

- `mm`：这里面保存了内存管理的信息，包括内存映射，虚存管理等内容。我的理解是这个结构体应该是用于进程内存的管理。
- `parent`：里面保存了进程的父进程的指针。我们在实验指导书知道，除了 `idle` 进程没有父进程，其余进程都存在父进程。这样有利于维护父进程对子进程的一些特殊操作。
- `context`：`context` 中保存了进程执行的上下文，也就是几个关键的寄存器的值。这些寄存器的值用于在进程切换中还原之前进程的运行状态。
- `tf`：`tf` 里保存了进程的中断帧。当进程从用户空间跳进内核空间的时候，进程的执行状态被保存在了中断帧中。
- `pgdir`：即页目录（Page Directory）的基址。在 RISC-V 架构中，CPU 通过 `satp` 寄存器找到当前页表的根节点，从而进行地址翻译。`pgdir` 字段保存的就是每个进程的页表根节点的物理地址。当进行进程切换时，内核需要将下一个要运行进程的 `pgdir` 值加载到 `satp` 寄存器中，这样才能正确地切换到新的地址空间。
- `kstack`：每个线程都有一个内核栈，并且位于内核地址空间的不同位置。对于内核线程，该栈就是运行时的程序使用的栈；而对于普通进程，该栈是发生特权级改变的时候使保存被打断的硬件信息用的栈。`kstack`记录了分配给该进程/线程的内核栈的位置。

我们主要通过下面这样来实现：

```

1 static struct proc_struct *
2 alloc_proc(void)
3 {
4     struct proc_struct *proc = kmalloc(sizeof(struct proc_struct));
5     if (proc != NULL)
6     {
7         // LAB4:EXERCISE1 2312323
8         proc->state = PROC_UNINIT;
9         proc->pid = -1;
10        proc->pgdir = boot_pgdir_pa;
11        proc->runs = 0;
12        proc->kstack = 0;
13        proc->need_resched = 0;
14        proc->parent = NULL;
15        proc->mm = NULL;
16        memset(&(proc->context), 0, sizeof(struct context));
17        proc->tf = NULL;
18        proc->flags = 0;
19        memset(proc->name, 0, PROC_NAME_LEN + 1);
20    }
21    return proc;
22 }

```

对于该结构体中的每个成员变量，我们初始化为默认的状态，包括

- 初始化进程状态为未初始化: `proc->state = PROC_UNINIT;`
- 初始化进程ID为-1: `proc->pid = -1;`
- 使用内核页目录表的基址: `proc->pgdir = boot_pgdir_pa;`
- 初始化运行次数为0: `proc->runs = 0;`
- 初始化内核栈地址为0: `proc->kstack = 0;`
- 初始化不需要重新调度: `proc->need_resched = 0;`
- 初始化父进程指针为NULL: `proc->parent = NULL;`
- 初始化内存管理结构为NULL: `proc->mm = NULL;`
- 初始化上下文结构，且全部设为0: `memset(&(proc->context), 0, sizeof(struct context));`
- 初始化陷阱帧（tf）指针为NULL: `proc->tf = NULL;`
- 初始化进程标志为0: `proc->flags = 0;`
- 初始化进程名称为空字符串: `memset(proc->name, 0, PROC_NAME_LEN + 1);`

问题

请说明proc_struct中 `struct context context` 和 `struct trapframe *tf` 成员变量含义和在本实验中的作用是啥？

struct context context

含义：存储进程在内核态运行时的关键寄存器状态，是进程切换（`switch_to`）的核心数据。

作用：1.保存与恢复内核态执行现场

2.初始化内核线程 / 子进程的执行起点

3.最小化切换开销

struct trapframe *tf

含义：指向一块内存区域，存储进程在触发中断、异常或系统调用时的完整寄存器状态（包括用户态和内核态的所有通用寄存器、状态寄存器 `sstatus`、程序计数器 `epc` 等）

- 作用：
1. 保存中断 / 异常发生时的全量状态
 2. 传递内核线程 / 子进程的启动参数与执行入口
 3. 衔接用户态与内核态

（三）练习2：分配并初始化一个进程控制块（需要编码）

编码

`do_fork()` 是 `ucore` 操作系统内核中进程/线程创建的核心函数，其核心功能是：以当前进程（父进程）为模板，创建一个新的子进程/线程，复制父进程的资源，并将子进程设为就绪态，最终返回子进程的 `PID` 或 `0`。

我们主要的实现方式如下所示：

第一步，调用 `alloc_proc`，首先获得一块用户信息块。

```
1 | if ((proc = alloc_proc()) == NULL) {  
2 |     goto fork_out;  
3 | }
```

第二步，为进程分配一个内核栈。

```
1 | if (setup_kstack(proc) != 0) {  
2 |     goto bad_fork_cleanup_proc;  
3 | }
```

第三步，复制原进程的内存管理信息到新进程

```
1 | if (copy_mm(clone_flags, proc) != 0) {  
2 |     goto bad_fork_cleanup_kstack;  
3 | }
```

第四步，复制原进程上下文到新进程

```
1 | copy_thread(proc, stack, tf);
```

第五步，将新进程添加到进程列表

```
1 | bool intr_flag;  
2 | local_intr_save(intr_flag); // 关中断，保护临界区  
3 | proc->pid = get_pid();  
4 | proc->parent = current;  
5 | hash_proc(proc);  
6 | list_add(&proc_list, &(proc->list_link));  
7 | nr_process++;  
8 | local_intr_restore(intr_flag);
```

第六步，唤醒新进程

```
1 wakeup_proc(proc);
```

第七步，返回新进程号

```
1 ret = proc->pid;
```

这七步是 `do_fork()` 创建子进程的核心流程，通过这七步我们可以实现从资源分配到激活就绪完成子进程全生命周期初始化。即先通过 `alloc_proc` 分配并初始化 `PCB`，再用 `setup_kstack` 分配独立内核栈，依据 `clone_flags` 经 `copy_mm` 复制或共享内存空间，通过 `copy_thread` 配置陷阱帧与内核上下文；随后关中断保护临界区，为子进程分配 `PID`、绑定父进程并注册到内核进程链表与哈希表，更新进程总数后开中断；最后通过 `wakeup_proc` 设子进程为就绪态，父进程返回子进程 `PID`，全程保障资源无泄漏、并发安全，实现进程/线程统一创建。

问题

请说明 `ucore` 是否做到给每个新 `fork` 的线程一个唯一的 `id`？请说明你的分析和理由

`ucore` 做到了给每个新 `fork` 的线程分配唯一 `ID (PID)`。分析与理由如下：通过 `do_fork()` 第 5 步的 `get_pid()` 函数，系统会从 1 到 `MAX_PID` 范围内分配未被占用的 `PID`，分配时遍历 `proc_list` 检查 `PID` 唯一性，若当前 `PID` 已被使用则递增重试，直至找到空闲 `PID`；且 `nr_process` 限制进程总数不超过 `MAX_PID`，结合 `MAX_PID > MAX_PROCESS` 的断言，确保不会出现 `PID` 耗尽导致的重复问题，同时新线程的 `PID` 会存入 `PCB` 并注册到哈希表，进一步保障后续查找与管理中 `PID` 的唯一性。

(四) 练习3：编写 `proc_run` 函数（需要编码）

编码

`proc_run` 是 `ucore` 内核的进程切换核心函数，作用是将指定进程 `proc` 调度到 `CPU` 上执行。其执行步骤主要为：先判断目标进程是否为当前运行进程，若不是则先通过 `local_intr_save` 禁用中断保障并发安全；随后保存当前进程 `prev`、指定目标进程 `next`，更新全局当前进程指针 `current` 为目标进程；再根据目标进程 `pgdir` 状态切换页表（若为用户进程，则加载自身页表。若为内核线程，则使用内核页表 `boot_pgdir_pa`），确保地址空间正确。接着通过 `switch_to` 完成 `prev` 与 `next` 的内核态上下文切换，最后用 `local_intr_restore` 恢复中断，实现进程的无缝切换与执行权移交。

```
1 void proc_run(struct proc_struct *proc)
2 {
3     if (proc != current)
4     {
5         // LAB4:EXERCISE3 2312796
6         /*
7          * Some Useful MACROS, Functions and DEFINES, you can use them in
8          * below implementation.
9          * MACROS or Functions:
10          * local_intr_save(): Disable interrupts
11          * local_intr_restore(): Enable Interrupts
12          * lsatp(): Modify the value of satp register
13          * switch_to(): Context switching between two
14          * processes
15          */
```

```

14     bool intr_flag;
15     struct proc_struct *prev = current; // 保存当前进程
16     struct proc_struct *next = proc;    // 设置要切换到进程
17     // 禁用中断
18     local_intr_save(intr_flag);
19     // 切换当前进程为要运行的进程
20     current = proc;
21     // 切换页表, 使用新进程的地址空间
22     // 注意: 对于内核线程, pgdir可能为NULL, 使用boot_pgdir_pa
23     if (proc->pgdir != 0) {
24         lsatp(proc->pgdir); // 加载新进程的页表到 satp 寄存器
25     } else {
26         lsatp(boot_pgdir_pa); // 使用内核页表
27     }
28     // 实现上下文切换
29     switch_to(&(prev->context), &(next->context));
30     // 允许中断
31     local_intr_restore(intr_flag);
32 }
33 }

```

问题

在本实验的执行过程中, 创建且运行了几个内核线程?

创建了两个内核线程, 分别是 `idleproc` (idle 线程) 和 `initproc` (init 线程)。

(五) 扩展练习Challenge

1. 说明语句 `local_intr_save(intr_flag); ... local_intr_restore(intr_flag);` 是如何实现开关中断的?

为了探究这两个语句实现开关中断的功能, 我们首先在代码中寻找这两个函数的声明。通过查找, 我们会发现, 这两个函数在 `sync.h` 中有如下声明:

```

1 #define local_intr_save(x) \
2     do {                    \
3         x = __intr_save(); \
4     } while (0)
5 #define local_intr_restore(x) __intr_restore(x);

```

然后, 在这个声明中, 我们可以看到这两个函数分别调用了两个静态的内联函数 `__intr_save()` 和 `__intr_restore(x)`。同样, 这两个函数也在 `sync.h` 中被定义, 如下所示:


```

1 static inline bool __intr_save(void) {
2     if (read_csr(sstatus) & SSTATUS_SIE) {
3         intr_disable();
4         return 1;
5     }
6     return 0;
7 }
8
9 static inline void __intr_restore(bool flag) {
10     if (flag) {
11         intr_enable();
12     }
13 }

```

我们可以看到 `__intr_save` 函数主要是根据函数 `read_csr(sstatus)` 的返回值与 `SSTATUS_SIE` 的值来进行判断。而 `__intr_restore` 函数则主要根据传入的 `flag` 值来进行处理。上面两个 `read_csr(sstatus)` 的返回值与 `SSTATUS_SIE` 的值，我们会在之后解释。我们在去寻找 `__intr_save()` 和 `__intr_restore(x)` 这两个函数的实现，我们会发现，这两个函数是在 `intr.c` 里实现的，如下所示：

```

1 #include <intr.h>
2 #include <riscv.h>
3
4 /* intr_enable - enable irq interrupt */
5 void intr_enable(void) { set_csr(sstatus, SSTATUS_SIE); }
6
7 /* intr_disable - disable irq interrupt */
8 void intr_disable(void) { clear_csr(sstatus, SSTATUS_SIE); }

```

在这里我们就可以清楚的看到这两个函数都是通过给 `sstatus` 寄存器的 `SSTATUS_SIE` 位进行置位来实现的功能。而我们在实验三中已经学习到了，`sstatus` 寄存器中的 `SIE` 位，是中断控制位，当 `SIE` 位 1 时，中断可以发生；当 `SIE` 为 0 时，中断不能发生。通过这样的一步步封装、调用，我们就能够实现中断的使能与控制。

总之，语句 `local_intr_save(intr_flag); ... local_intr_restore(intr_flag);` 是通过控制 `sstatus` 寄存器里的 `SIE` 位从而实现中断的开关。

1. `get_pte()` 函数中有两段形式类似的代码，结合 `sv32`，`sv39`，`sv48` 的异同，解释这两段代码为什么如此相像。

`sv32`，`sv39`，`sv48` 的共性是**多级页表分层映射**，无论级数（32 位 `SV32` 为 2 级、64 位 `SV39` 为 3 级、`SV48` 为 4 级），本质都是将线性地址（LA）按页表级数拆分为多个索引（如 `SV32` 拆为 `PDX1/PDX0/PTX`，`SV39` 拆为 `PDX2/PDX1/PDX0/PTX`），每级索引对应一级页表，需逐层查找上级页表项，若不存在则创建，最终定位到最末级 PTE。

代码分为两段处理，其中，第一段是处理**第一级页目录项（PDX1 索引对应的 PDE）**，即检查该 PDE 是否有效（`PTE_V`），若无效则创建对应的页表页；第二段是处理**第二级页目录项（PDX0 索引对应的 PDE）**，处理逻辑与第一段代码段相同，只是索引层级有所不同。

2. 目前 `get_pte()` 函数将页表项的查找和页表项的分配合并在一个函数里，你认为这种写法好吗？有没有必要把两个功能拆开？

目前 `get_pte` 函数的封装非常良好，没有必要拆开，这样良好的封装主要得益于参数中的 `create` 标志以及具体实现中的边查找，边分配，若不允许分配返回 `NULL` 的设计。

我们以大大页(Giga-Page)的查找为例子:我们首先对传入参数la的大大页对应位进行查找获得其pte在这里就已经完成了这一部分的查找工作, 查找完后检查其valid标志位, 仅在不允许创建, 或允许创建 (此处巧妙地利用了c语言对或关系表达式的短路求值(Short-Circuit Evaluation)评估逻辑, 仅在允许create时alloc_page) 但创建失败 (page=alloc_page()==NULL) 时, 直接返回NULL, 否则完成了分配, 并进行必要的初始化。大页的查找逻辑与上述类似。

基于如上的查找与分配逻辑, 对于该接口, 我们在使用时他就已经很好地在create标志设置不同的情况下, 分饰了查找和分配的功能, 只是在分配时, 利用了返回值把页表项的指针返回回去作为一个附加信息, 即使将两个功能分开设计于两个接口, 我认为在设计分配接口时也应当出于返回必要信息的设计原则将pte指针作为返回值返回, 那么分配接口几乎完全覆盖了查找接口的功能, 反倒出现了冗余的设计。而目前的设计既精简了接口, 通过同一个接口的开关位控制了两层功能, 还保证了分配操作的原子性及分配完成后应当返回必要信息的接口提供原则, 不必要再行冗余的拆分。

五、运行结果

在编写完练习的代码后, 我们使用 `make qemu` 尝试运行, 我们可以看到下面的结果:

```
yzx@yzx:~/桌面/labcode/原始代码/lab4$ make qemu
Base: 0x0000000080000000
Size: 0x0000000080000000 (128 MB)
End: 0x0000000087ffffff
DTB init completed
(THU.CST) os is loading ...

Special kernel symbols:
entry 0xc020004a (virtual)
etext 0xc0203edc (virtual)
edata 0xc0209030 (virtual)
end 0xc020d4ec (virtual)
Kernel executable memory footprint: 54KB
memory management: default_pmm_manager
physical memory map:
memory: 0x08000000, [0x80000000, 0x87ffffff].
vapaoffset is 18446744070488326144
check_alloc_page() succeeded!
check_pgdir() succeeded!
check_boot_pgdir() succeeded!
use SLOB allocator
kmalloc_init() succeeded!
check_vma_struct() succeeded!
check_vmm() succeeded.
alloc_proc() correct!
++ setup timer interrupts
this initproc, pid = 1, name = "init"
To U: "Hello world!!".
To U: "en., Bye, Bye. :)"
kernel panic at kern/process/proc.c:402:
process exit!!.

Welcome to the kernel debug monitor!!
Type 'help' for a list of commands.
```

然后我们使用 `make grade` 来测试得分, 结果如下所示:

```

yzx@yzx:~/桌面/labcode/原始代码/lab4$ make grade
riscv64-unknown-elf-ld: removing unused section '.text.pic_enable' in file 'obj/kern/driver/picirq.o'
riscv64-unknown-elf-ld: removing unused section '.text.slob_init' in file 'obj/kern/mm/kmalloc.o'
riscv64-unknown-elf-ld: removing unused section '.text.slob_allocated' in file 'obj/kern/mm/kmalloc.o'
riscv64-unknown-elf-ld: removing unused section '.text.kallocated' in file 'obj/kern/mm/kmalloc.o'
riscv64-unknown-elf-ld: removing unused section '.text.ksize' in file 'obj/kern/mm/kmalloc.o'
riscv64-unknown-elf-ld: removing unused section '.text.tlb_invalidate' in file 'obj/kern/mm/pmm.o'
riscv64-unknown-elf-ld: removing unused section '.rodata.print_vma.str1.8' in file 'obj/kern/mm/vmm.o'
riscv64-unknown-elf-ld: removing unused section '.text.print_vma' in file 'obj/kern/mm/vmm.o'
riscv64-unknown-elf-ld: removing unused section '.rodata.print_mm.str1.8' in file 'obj/kern/mm/vmm.o'
riscv64-unknown-elf-ld: removing unused section '.text.print_mm' in file 'obj/kern/mm/vmm.o'
riscv64-unknown-elf-ld: removing unused section '.text.mm_create' in file 'obj/kern/mm/vmm.o'
riscv64-unknown-elf-ld: removing unused section '.text.vma_create' in file 'obj/kern/mm/vmm.o'
riscv64-unknown-elf-ld: removing unused section '.text.mm_destroy' in file 'obj/kern/mm/vmm.o'
riscv64-unknown-elf-ld: removing unused section '.text.set_proc_name' in file 'obj/kern/process/proc.o'
riscv64-unknown-elf-ld: removing unused section '.text.get_proc_name' in file 'obj/kern/process/proc.o'
riscv64-unknown-elf-ld: removing unused section '.text.find_proc' in file 'obj/kern/process/proc.o'
riscv64-unknown-elf-ld: removing unused section '.text.sprintf' in file 'obj/libs/printfmt.o'
riscv64-unknown-elf-ld: removing unused section '.text.snprintf' in file 'obj/libs/printfmt.o'
riscv64-unknown-elf-ld: removing unused section '.text.vsnprintf' in file 'obj/libs/printfmt.o'
riscv64-unknown-elf-ld: removing unused section '.text.strncpy' in file 'obj/libs/string.o'
riscv64-unknown-elf-ld: removing unused section '.text.strfind' in file 'obj/libs/string.o'
riscv64-unknown-elf-ld: removing unused section '.text.strtol' in file 'obj/libs/string.o'
riscv64-unknown-elf-ld: removing unused section '.text.memmove' in file 'obj/libs/string.o'
gmake[1]: 进入目录"/home/yzx/桌面/labcode/原始代码/lab4" + cc kern/init/entry.S + cc kern/init/init.c + cc kern/libs/readlin
e.c + cc kern/libs/stdio.c + cc kern/debug/kdebug.c + cc kern/debug/kmonitor.c + cc kern/debug/panic.c + cc kern/driver/cloc
k.c + cc kern/driver/console.c + cc kern/driver/dtb.c + cc kern/driver/intr.c + cc kern/driver/picirq.c + cc kern/trap/trap.
c + cc kern/trap/trapentry.S + cc kern/mm/default_pmm.c + cc kern/mm/kmalloc.c + cc kern/mm/pmm.c + cc kern/mm/vmm.c + cc ke
rn/process/entry.S + cc kern/process/proc.c + cc kern/process/switch.S + cc kern/schedule/sched.c + cc kern/hash.c + cc libs
/printfmt.c + cc libs/string.c + ld bin/kernel riscv64-unknown-elf-objcopy bin/kernel --strip-all -O binary bin/ucore.img gm
ake[1]: 离开目录"/home/yzx/桌面/labcode/原始代码/lab4"
-check alloc proc: OK
-check initproc: OK
Total Score: 30/30
yzx@yzx:~/桌面/labcode/原始代码/lab4$

```

我们可以看到我们的得分为30/30，说明我们这次实验已经成功完成了。

六、知识点总结

1. 操作系统中内核线程与用户进程的区别

在本次实验中，我们主要接触和涉及了内核线程和用户进程两种进程类型，对他们的正确认知有裨益于对线程管理的进一步实现。

其中，内核线程是由操作系统内核直接支持和管理的线程，其在创建、销毁及切换过程中，会造成较大的开销，所有内核进程都在内核的内存空间中运行，不同的内核线程之间相互可见，因此它们之间的通信相对高效，但与此同时也带来了一些问题，如对同一块数据访问尤其是写操作时的并发问题，这种时候往往需要使用良好的锁结构来避免冲突；

而用户进程在用户态和内核态之间切换，通过一些创建、同步线程的API予以实现，具有较高的执行效率和较低的创建及切换开销，拥有独立的用户虚拟地址空间，高度隔离，不同的用户进程之间彼此不可见，受到操作系统的严格保护。

2. 线程控制块及其关键字段

为有序地监控和追踪不同进程的状态，我认为有必要再次梳理本次实验过程中的关键数据结构 `proc_struct` (线程控制块) 的一些重要属性。

`pgdir` - page directory, Base Address of Page Directory Table: 页目录基址，即对应进程的页表根，每当一个进程被加载时，会将其中的值赋给进程空间中的 `satp`，为进程营造可以访问整个机器的内存资源的假象。

`kstack` - kernel stack, 内核栈，对于内核线程，`kstack` 指向运行时程序使用的栈，而对于普通线程，`kstack` 所指向的栈将在特权级改变时被使用，用于保存被打断时的硬件信息。在本次实验中，内存栈被分配的空间较小，仅2个物理页(8MB)，因此在使用时应当紧凑地分布其中的数据，避免存放过大的数据。在进程创建或切换时，将根据内存栈中的值设置进程的 `tss` (task status segment, 任务状态段)。

status-线程状态标志位，在本次实验的架构中，线程有四种不同的状态，分别为PROC_UINIT、PROC_RUNNABLE、PROC_SLEEPING和PROC_ZOMBIE，这四个状态分别标志着当前进程为未初始化、可运行、休眠及僵尸态，其中PROC_RUNNABLE状态在本次实验中有较高的存在感，本次实验中在proc_init函数中，我们会对init_main进程的status状态位进行设置为PROC_RUNNABLE，紧随其后，cpu_idle函数会对原来的idle_main空闲进程进行reschedule，在reschedule函数中会依据该标志位，找到首个状态为PROC_RUNNABLE的进程并将其作为switch_to函数的to参数，即即将转换的进程，因此，该标志位的正确设置对于进程调度及切换工作尤为重要。

parent-父进程指针，在进程控制块中调用do_fork函数的线程，即切换向目的进程的原进程，由于本次实验中的parent标志位设置依据以上逻辑，故我们这样描述parent指针。值得一提，在本次实验中除idle_main进程外，别的进程的该属性均不应为空，由于idle_main进程是启动时的第零个进程，故其父进程沿用proc_alloc函数中初始化的NULL。

need_resched-需要调度标志位，主要在cpu_idle函数中作为调度使能条件被消费。

list_link-线程双向链表连接属性。指向存储进程信息的双向链表的下一个进程的重要字段，在本次实验中主要用于以idle_main线程为起点，遍历寻找到下一个可以运行的线程，即为init_main。

tf-进程的trapframe，巧妙地沿用了陷阱处理中的上下文保存设计，作为进程切换过程中的辅助信息，这一部分信息的使用我们会在后面的进程切换部分详细介绍。

此外，还有一些在本次实验中虽然存在感较低，但是仍然至关重要的属性，如 **PID** (Process Identifier,进程标识,由全局函数get_pid唯一分配,不应出现重复)、**runs** (已运行次数)、**mm** (内存管理结构)、**proc_name**。

3.进程初始化与切换

在本次实验过程中，进程切换模块在很大程度上展现了操作系统设计的艺术，考虑到进程切换过程中的上下文切换过程与终端处理过程中的SAVE_ALL与RESTORE_ALL的期望效果高度相似，所以在实现的过程中，我们重用了中断处理过程中的tf结构体及RESTALL_ALL函数（当然，我们进一步以forkrets标签封装了它），将寄存器状态的保存与切换工作在一定程度上向forrets委托。

在进程初始化的初期，主要在proc_init函数中调用了kernel_thread函数，在其中完成进程入口函数的初始化，此后，其返回时调用的forkret完成了大量的进程初始化工作，其中尤为重要的一项就是进行了copy_thread函数的调用，其不仅完成了父进程的tf赋值，使得该进程在运行时使用与父进程相同的资源，此外其调用的proc->context.ra = (uintptr_t)forkret;设置了返回地址，使得ret时自动进行进程状态的回复，减少了不必要的显式调用。

完成上述工作后，方能在proc_run函数中真正地调用switch_to函数，进行部分寄存器的设置。因为在调用函数的过程中，编译器会自动维护调用者寄存器(caller-saved registers)的状态，所以我们只需要手动地维护好被调用者寄存器(callee-saved register,ra,sp,s0-s11,14 in total)即可。

4.进程调度

在本次实验过程中，我们实现了由初始空闲进程idle_main向ini_main进程的切换，由idle_main向init_main函数的切换操作是有schedule进程调度函数负责的，得益于进程双向链表*proc_link及各个进程中的list_link属性的维护，这样的遍历操作成为可能。

实验中的进程调度逻辑大致如下：首先确定遍历逻辑，若传入的被schedule(由proc_struct中的need_resched属性决定)进程为双向链表起点节点，即idle_init函数，则从头开始遍历，若非起点进程，则从该进程开始，利用list_link属性进行迭代。迭代的目的是找到status为PROC_RUNNABLE的进程，作为将被switch_to的进程。若无这样的进程，则以idle_init进程作为兜底，重新执行该进程，以上则为进程调度过程的大致逻辑。

5.本次实验中的虚拟内存管理

本次实验中的虚拟内存管理工作主要在page_insert和page_remove两个函数中实现，并在check_pgdir中对其进行调用，从而模拟并检查这两个函数是否实现了良好的内存分配和管理功能，具体的检测过程如下：

在检查过程中，主要进行了两个物理页面通过多级页表映射并删除映射检查功能是否实现，在该函数中，我们先后取得了两个物理页面，并将其映射到0x0和4096两个虚拟地址，对于第一个页，我们简单地使用两个断言：

```
1 | pte_t *ptep;  
2 | assert((ptep = get_pte(boot_pgdir_va, 0x0, 0)) != NULL);  
3 | assert(pte2page(*ptep) == p1)
```

对其页表项是否正确分配并(广义地)指向p1，而对于p2，我们特意将其映射向PGSIZE对应的位置，从而确保两个页的位置相邻。此后，我们通过如下两行代码及一个断言：

```
1 | ptep = (pte_t *)KADDR(PDE_ADDR(boot_pgdir_va[0]));  
2 | ptep = (pte_t *)KADDR(PDE_ADDR(ptep[0])) + 1; //get next pte  
3 | assert(get_pte(boot_pgdir_va, PGSIZE, 0) == ptep)
```

手动计算第二页从而检查页表遍历机制

然后以主动覆盖的形式，检查映射重叠时，被覆盖的p2页的ref是否正常取消，且建立多映射的p1是否正常地增加了映射技术，以一下一行代码和两行断言实现：

```
1 | assert(page_insert(boot_pgdir_va, p1, PGSIZE, 0) == 0);  
2 | assert(page_ref(p1) == 2); //0x0、PGSIZE->p1  
3 | assert(page_ref(p2) == 0); //original PGSIZE overlapped
```

然后是一些相对次要的权限设置检查，此处略去。